

BitNet v2: Native 4-bit Activations with Hadamard Transformation for 1-bit LLMs

Hongyu Wang* Shuming Ma* Furu Wei[◇]
<https://aka.ms/GeneralAI>

Abstract

Efficient deployment of 1-bit Large Language Models (LLMs) is hindered by activation outliers, which complicate quantization to low bit-widths. We introduce BitNet v2, a novel framework enabling native 4-bit activation quantization for 1-bit LLMs. To tackle outliers in attention and feed-forward network activations, we propose $\mathcal{H}$ -BitLinear, a module applying an online Hadamard transformation prior to activation quantization. This transformation smooths sharp activation distributions into more Gaussian-like forms, suitable for low-bit representation. Experiments show BitNet v2 trained from scratch with 8-bit activations matches BitNet b1.58 performance. Crucially, BitNet v2 achieves minimal performance degradation when trained with native 4-bit activations, significantly reducing memory footprint and computational cost for batched inference.

“

There’s Plenty of Room at the Bottom.

”

Richard Feynman

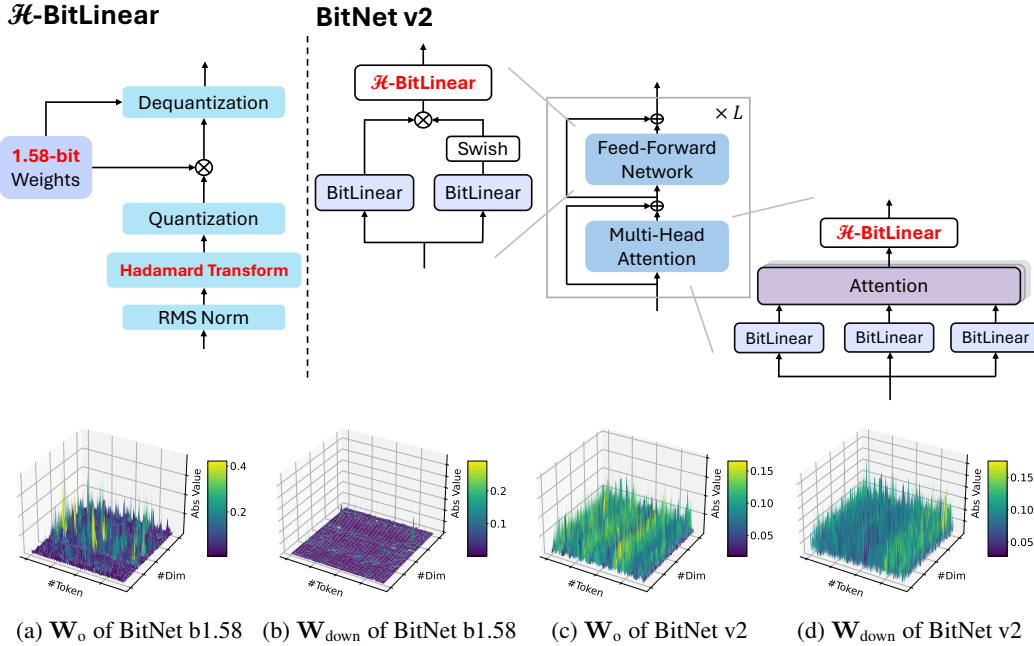


Figure 1: **Top:** Overview of BitNet v2 and $\mathcal{H}$ -BitLinear. **Bottom:** The distribution of the activation of output projection $\mathbf{W}_o$ in attention and down projection $\mathbf{W}_{\text{down}}$ in FFN. BitNet v2 utilizes $\mathcal{H}$ -BitLinear to eliminate the large amount of outlier channels in the intermediate states. The Hadamard transformation reshapes the original sharp distribution into a more Gaussian-like form.

* Equal contribution. [◇] Corresponding author. S. Ma and F. Wei are with Microsoft Research. H. Wang is with University of Chinese Academy of Sciences.

1 Introduction

The field of deep learning is rapidly embracing quantization-aware training and low-bit inference, driven by hardware advancements like next-generation GPUs (e.g., GB200) offering native support for 4-bit computations. This promises significant efficiency gains for large-scale models. Pioneering work like BitNet b1.58 [MWM⁺24] demonstrated that 1.58-bit LLMs can match full-precision performance while drastically reducing inference costs (latency, memory, throughput, energy) [WZS⁺25]. However, while BitNet b1.58 quantizes weights to 1.58 bits, alleviating memory bandwidth bottlenecks, it retains 8-bit activations. This reliance on 8-bit precision prevents these models from fully leveraging the 4-bit computational capabilities of emerging hardware, shifting the efficiency bottleneck towards computation itself.

Achieving lower bit-width activations is crucial for maximizing hardware utilization, particularly for efficient kernel design in batched inference scenarios. Research [WMW24, LPC⁺24] highlights a key challenge: the non-uniform distribution of activations within LLMs. While inputs to attention and feed-forward network (FFN) layers often exhibit Gaussian-like distributions amenable to quantization, their intermediate states (outputs before final projection) contain significant outliers, hindering aggressive low-bit quantization. BitNet a4.8 [WMW24] attempted to address this by selectively using 4-bit quantization for inputs and 8-bit sparsification for intermediate states. While achieving minimal performance loss compared to 8-bit activations, sparsification is less suited for maximizing throughput in batched inference, where dense computations are often preferred for hardware efficiency.

To bridge this gap and unlock the full potential of 4-bit computation for 1.58-bit LLMs, we introduce **BitNet v2**. Our framework enables *native* 4-bit activations across the model. The core innovation is **$\mathcal{H}$ -BitLinear**, a novel linear layer replacing the standard output projections in attention and down projections in FFNs. $\mathcal{H}$ -BitLinear applies an online Hadamard transformation before activation quantization. This strategically reshapes the sharp, outlier-prone distributions of intermediate states into more manageable, Gaussian-like forms, significantly reducing the impact of outliers in 1.58-bit models. We train BitNet v2 from scratch using 8-bit activations, achieving negligible performance loss compared to BitNet b1.58 [MWM⁺24]. Subsequently, the model can be efficiently fine-tuned with a small amount of data to operate with native 4-bit activations. Extensive experiments demonstrate that our 4-bit BitNet v2 variant achieves performance comparable to BitNet a4.8 while offering superior computational efficiency for batched inference scenarios.

2 BitNet v2: Native 4-bit Activations

We illustrate the architecture of BitNet v2 in Figure 1. We implement BitNet v2 using LLaMA-like components, including RMS normalization [ZS19], SwishGLU [Sha20] and removing all bias. Compared to BitNet [WMD⁺23], we use $\mathcal{H}$ -BitLinear for $\mathbf{W}_o$ in attention and $\mathbf{W}_{down}$ in FFN layers to deal with outlier channels of intermediate states. BitNet v2 is trained with 1.58-bit weights and INT8 activations from scratch, then continue-trained with INT4 activations for all linear layers except input/output embedding.

2.1 $\mathcal{H}$ -BitLinear

Following [WMD⁺23, MWM⁺24], as for weight quantization, we use per-tensor *absmean* function to quantize the weights into ternary values, i.e., $\{-1, 0, 1\}$:

$$Q_w(\mathbf{W}) = \alpha \text{RoundClip}\left(\frac{\mathbf{W}}{\alpha + \epsilon}, -1, 1\right), \alpha = \text{mean}(|\mathbf{W}|) \quad (1)$$

$$\text{RoundClip}(X, a, b) = \min(\max(\text{round}(X), a), b) \quad (2)$$

For low-bit activations, previous works [WMW24, LPC⁺24] have shown that the distributions of the inputs to the attention and feed-forward-network layers (i.e., the activations of $\mathbf{W}_{qkv}$ and $\mathbf{W}_{up, gate}$) tend to exhibit a Gaussian-like shape, while the intermediates states (i.e., the activations of $\mathbf{W}_o$ and $\mathbf{W}_{down}$) have more outlier channels and massive amount of entries around zero.

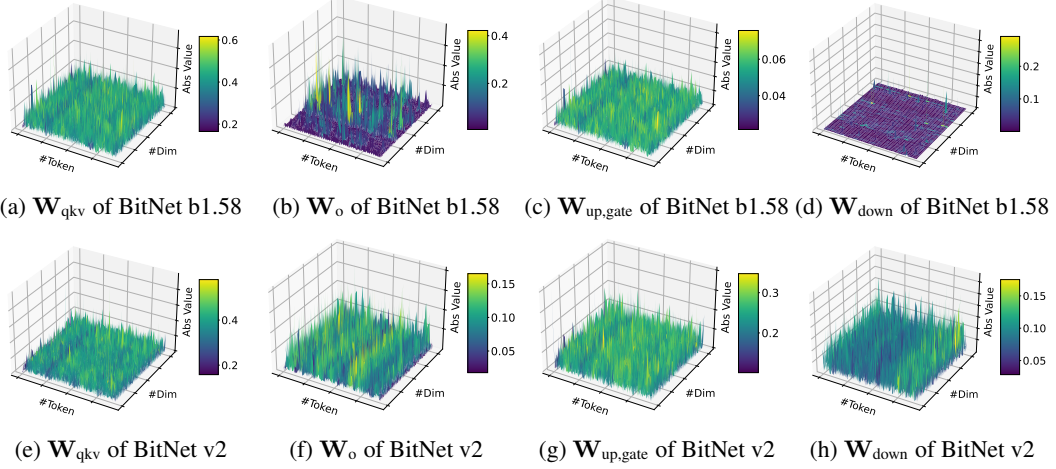


Figure 2: The activation distribution of BitNet b1.58 and BitNet v2 with 8-bit activations.

Therefore, we introduce $\mathcal{H}$ -BitLinear for $\mathbf{W}_{\text{o}}$ in attention and $\mathbf{W}_{\text{down}}$ in FFN layers. $\mathcal{H}$ -BitLinear employs a Hadamard transformation before activation quantization to first reduce the number of outlier channels. The Hadamard transformation satisfies that:

$$\text{Hadamard}(\mathbf{X}) = \mathbf{H}_{\mathbf{m}} \mathbf{X} \quad (3)$$

$$\mathbf{H}_{\mathbf{m}} = \frac{1}{\sqrt{2}} \begin{pmatrix} \mathbf{H}_{\mathbf{m}-1} & \mathbf{H}_{\mathbf{m}-1} \\ \mathbf{H}_{\mathbf{m}-1} & -\mathbf{H}_{\mathbf{m}-1} \end{pmatrix}, \mathbf{H}_0 = (1) \quad (4)$$

where $\mathbf{H}_{\mathbf{m}}$ is a $2^m \times 2^m$ matrix and $\mathbf{X} \in \mathbb{R}^n, n = 2^m$. We use fast-hadamard-transform² to perform the matrix multiplication, which has $\mathcal{O}(n \log n)$ computational complexity.

As shown in Figure 2 and Figure 3, with Hadamard transformation, the distribution of the intermediate states becomes closer to a Gaussian-like distribution, which significantly reduces the number of outliers and make it more suitable for INT4 quantization.

For 8-bit and 4-bit activations, we adopt per-token *absmax* and *absmean* function, respectively. The activation quantization can be formulated as:

$$\mathbf{Q}_{\text{INT8}}(\mathbf{X}) = \frac{\gamma}{127} \text{RoundClip}\left(\frac{127}{\gamma + \epsilon} \mathbf{X}, -128, 127\right), \gamma = \max(|\mathbf{X}|) \quad (5)$$

$$\mathbf{Q}_{\text{INT4}}(\mathbf{X}) = \frac{\beta}{\sqrt{7}} \text{RoundClip}\left(\frac{\sqrt{7}}{\beta + \epsilon} \mathbf{X}, -8, 7\right), \beta = \text{mean}(|\mathbf{X}|) \quad (6)$$

Above all, the matrix multiplication of $\mathcal{H}$ -BitLinear can be written as:

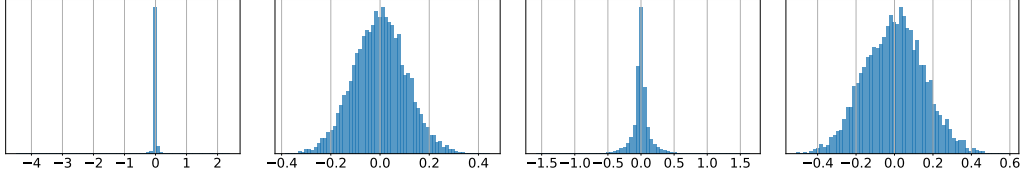
$$\mathbf{Y} = \mathbf{Q}_w(\mathbf{W}) \cdot \mathbf{Q}_{\text{INT8/4}}(\mathbf{X}_{\text{r}}), \mathbf{X}_{\text{r}} = \text{Hadamard}(\text{LN}(\mathbf{X})) \quad (7)$$

where LN denotes the layer normalization.

2.2 Training

Following [WMD⁺23], we employ the straight-through estimator (STE) [BLC13] for gradient approximation and use mixed-precision training to update the parameters. During backward propagation,

²<https://github.com/Dao-AILab/fast-hadamard-transform>



(a) $\mathbf{W}_{\text{down}}$ of BitNet b1.58 (b) $\mathbf{W}_{\text{down}}$ of BitNet v2 (c) $\mathbf{W}_o$ of BitNet b1.58 (d) $\mathbf{W}_o$ of BitNet v2

Figure 3: The activation distribution of $\mathbf{W}_{\text{down}}$ in FFN and $\mathbf{W}_o$ in attention of BitNet b1.58 and BitNet v2 with 8-bit activations.

Models	Size	PPL $\downarrow$	ARCC $\uparrow$	ARCE $\uparrow$	HS $\uparrow$	PQ $\uparrow$	WGe $\uparrow$	LBA $\uparrow$	Avg $\uparrow$
BitNet b1.58	400M	13.37	24.32	43.01	39.51	64.91	51.93	45.51	44.87
BitNet a4.8		13.61	24.15	41.75	39.48	65.18	53.59	44.34	44.75
BitNet v2 (a8)		13.50	23.29	43.06	39.06	64.74	50.59	45.26	44.33
BitNet v2 (a4)		13.78	23.29	41.46	38.33	65.45	50.59	44.56	43.95
BitNet b1.58	1.3B	11.02	27.90	49.58	48.85	69.80	55.80	54.12	51.01
BitNet a4.8		11.15	27.47	49.20	48.72	69.64	56.51	53.85	50.90
BitNet v2 (a8)		11.14	27.90	49.96	48.37	69.42	57.22	54.14	51.17
BitNet v2 (a4)		11.33	27.56	49.58	48.00	68.23	55.49	53.58	50.41
BitNet b1.58	3B	9.71	28.84	54.80	56.39	71.44	59.35	60.47	55.22
BitNet a4.8		9.80	29.01	55.01	55.92	71.76	59.59	59.85	55.19
BitNet v2 (a8)		9.72	30.55	55.56	57.19	71.33	58.72	60.90	55.71
BitNet v2 (a4)		9.85	28.92	55.01	56.59	71.65	59.67	60.74	55.43
BitNet b1.58	7B	9.09	31.74	59.51	61.49	74.37	59.98	61.63	58.12
BitNet a4.8		9.16	31.91	59.09	61.06	74.16	59.67	61.54	57.91
BitNet v2 (a8)		9.14	32.94	58.54	61.08	74.10	61.48	64.22	58.73
BitNet v2 (a4)		9.24	32.42	58.00	60.71	74.27	60.85	63.52	58.30

Table 1: Perplexity and results of BitNet v2, BitNet a4.8 and BitNet b1.58 on the end tasks.

we bypass the non-differentiable functions in quantization. To support mixed-precision training, we maintain a full-precision latent weight to accumulate parameter updates.

For the backward pass through the Hadamard transformation, we apply the transformation to the gradients as well, leveraging the orthogonality of the transformation matrix $\mathbf{H}_m$. Specifically, the backward propagation is formulated as:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{X}} = \text{Hadamard}\left(\frac{\partial \mathcal{L}}{\partial \text{Hadamard}(\mathbf{X})}\right) \quad (8)$$

Similar to BitNet a4.8, BitNet v2 with 4-bit activations can be continue-trained from its 8-bit activation counterpart using a small number of training tokens, while incurring negligible performance loss. The optimizer states are reused for continue-training.

3 Experiments

We compared BitNet v2 to BitNet b1.58 and BitNet a4.8 of various model sizes. All models were trained with 1.58-bit weights. BitNet b1.58 has fully INT8 activations for all linear layers. BitNet a4.8 is continue-trained from BitNet b1.58 with a hybrid quantization and sparsification for activations, where the inputs to the sublayers are quantized into 4-bit integers and the intermediate states uses top- K sparsification [WMW⁺24] and squared ReLU.

We adopted the two-stage weight decay and learning rate scheduling following the training recipe of BitNet b1.58 [MWM⁺24]. All models were trained with 100B tokens from the RedPajama

Models	Size	ARCCe $\uparrow$	ARCCe $\uparrow$	HS $\uparrow$	PQ $\uparrow$	WGe $\uparrow$	LBA $\uparrow$	Avg $\uparrow$
BitNet v2 (a8)	3B	30.55	55.56	57.19	71.33	58.72	60.90	55.71
w/ 4-bit KV		29.52	55.18	57.17	70.95	58.56	60.84	55.37
w/ 4-bit QKV		30.63	55.22	57.15	71.16	58.96	60.49	55.60
w/ 4-bit Q, 3-bit KV		29.69	55.22	56.22	71.49	57.62	59.01	54.88
BitNet v2 (a8)	7B	32.94	58.54	61.08	74.10	61.48	64.22	58.73
w/ 4-bit KV		33.02	58.67	61.04	73.61	61.88	64.06	58.71
w/ 4-bit QKV		32.76	58.46	61.01	74.10	60.85	63.87	58.51
w/ 4-bit Q, 3-bit KV		32.51	58.29	60.85	73.39	60.77	62.99	58.13

Table 2: The zero-shot accuracy of BitNet v2 with 8-bit activations and QKV states varying bit-widths on the end tasks.

Models	Size	ARCCe $\uparrow$	ARCCe $\uparrow$	HS $\uparrow$	PQ $\uparrow$	WGe $\uparrow$	LBA $\uparrow$	Avg $\uparrow$
BitNet v2 (a4)	3B	28.92	55.01	56.59	71.65	59.67	60.74	55.43
w/ 4-bit KV		29.52	54.46	56.36	71.49	58.17	60.14	55.02
w/ 4-bit QKV		28.58	55.43	56.32	71.16	57.93	60.70	55.02
w/ 4-bit Q, 3-bit KV		29.18	55.51	55.85	71.60	58.41	59.54	55.02
BitNet v2 (a4)	7B	32.42	58.00	60.71	74.27	60.85	63.52	58.30
w/ 4-bit KV		32.94	58.12	60.33	74.21	61.01	63.65	58.38
w/ 4-bit QKV		33.11	57.91	60.78	74.05	61.17	62.93	58.33
w/ 4-bit Q, 3-bit KV		32.08	57.95	60.29	73.23	59.59	62.97	57.69

Table 3: The zero-shot accuracy of BitNet v2 with 4-bit activations and QKV states varying bit-widths on the end tasks.

dataset [Com23] to ensure a fair comparison. For BitNet v2 (a4) and BitNet a4.8, we first trained the model with 8-bit activations for 95B tokens. Then we reused the optimizer states and continue-train the model with 4-bit activations for 5B tokens. More details can be found in the Appendix B.

We evaluated the zero-shot accuracy for these models on a range of language tasks using the *lm-evaluation-harness* toolkit [GTA⁺24], including ARC-Easy (ARCE) [YBS19], ARC-Challenge (ARCC) [YBS19], Hellaswag (HS) [ZHB⁺19], Winogrande (WGe) [SBBC20] and PIQA (PQ) [BZB⁺19] and LAMBADA (LBA) [PKL⁺16]. We also reported the perplexity on the validation set of C4 [RSR⁺19] dataset.

3.1 Main Results

We present the detailed results of BitNet v2 and the baselines in Table 1. Introducing the Hadamard transformation before the quantization in attention and FFN layers results in minimal perplexity degradation. For 8-bit activations, BitNet v2 surpasses BitNet b1.58 with an average accuracy improvement of 0.16%, 0.49%, and 0.61% on end tasks for the 1.3B, 3B, and 7B model sizes, respectively. Additionally, BitNet v2 enables native 4-bit activations across all linear layers, enhancing efficiency for batched inference. With INT4 activations, BitNet v2 achieves perplexity comparable to BitNet a4.8 while demonstrating superior performance on downstream tasks for the 3B and 7B models.

Table 2 and Table 3 summarize detailed results of BitNet v2 (a8) and BitNet v2 (a4) with low-bit attention, respectively. We adopt post-RoPE quantization for QKV states. The QKV heads were directly quantized to unsigned integers using the absmax function, without the need of any calibration dataset. We retain the KV heads of [BOS] token as 8-bit precision. As shown in Table 2 and Table 3, BitNet v2 with 3-bit KV Cache achieves accuracy comparable to its counterpart with full-precision KV cache in 3B and 7B models.

3.2 Comparison with Post-Training Quantization.

We compared BitNet v2 (a4) with post-training quantization baselines, including SpinQuant [LZF⁺24] and QuaRot [AMC⁺24], in 1.3B models. QuaRot employs randomized Hadamard

Models	PPL↓	ARCC↑	ARCE↑	HS↑	PQ↑	WGe↑	LBA↑	Avg↑
<i>w/o fusing rotary matrix to $W_{qkv,up,gate}$</i>								
QuaRot	13.52	26.28	47.43	45.92	65.89	51.46	42.34	46.55
SpinQuant	13.52	25.60	47.35	45.52	67.25	52.49	42.52	46.79
QuaRot	20.83	24.74	40.78	40.54	62.89	49.33	36.89	42.53
SpinQuant	19.80	24.74	40.19	40.77	62.73	52.09	39.24	43.29
BitNet v2 (a4)	11.33	27.56	49.58	48.00	68.23	55.49	53.58	50.41

Table 4: Perplexity and zero-shot accuracy of BitNet v2, QuaRot and SpinQuant on the end tasks.

Methods	#Bits	1.3B		3B	
		Acc.↑	PPL↓	Acc.↑	PPL↓
No rotation	W1.58A8	51.01	11.02	55.22	9.71
Weight & activation rotation		50.47	11.14	55.55	9.69
Activation rotation		51.16	11.14	55.71	9.72
No rotation	W1.58A4	diverged		diverged	
Weight & activation rotation		50.09	11.33	54.98	9.81
Activation rotation		50.41	11.33	55.43	9.85

Table 5: Ablations on the Hadamard transformation of $\mathcal{H}$ -BitLinear across various sizes.

transformations to mitigate outlier features, while SpinQuant uses the learnable rotary matrix. Then they adopt GPTQ and *absmax* function to quantize the weight and activations into 4-bit, respectively. Since the weights of BitNet b1.58 were already trained to be ternary values from scratch, we adopted *absmean* function used in the training of BitNet rather than GPTQ for weight quantization. For activation quantization of the baselines, we retained rotary transformations for activations as an online operation; Furthermore, we removed the fusion of RMS normalization scales into projections. Following [LZF⁺24], we tuned the rotation of SpinQuant with 800 samples from WikiText2 dataset.

We report the results of BitNet v2 (a4) and BitNet b1.58 with SpinQuant and QuaRot in Table 4. All models were quantized to 1.58-bit weight and 4-bit activations. BitNet v2 (a4) significantly surpasses these baselines in terms of perplexity on valid set of C4 and accuracy on downstream tasks. Additionally, we observed that ternary models are more sensitive to the fusion of rotary matrix and latent weights. Specifically, we removed rotary matrix fusion for W_{qkv} in attention modules and $W_{up,gate}$ in FFNs, i.e., using $Q_w(W)R^T$ rather than $Q_w(WR^T)$. Removing this fusion notably enhances baseline performance: the perplexity decreases from 19.80 to 13.52 for SpinQuant. However, they still trails substantially behind BitNet v2 (a4). Moreover, this adjustment forces these projections to revert to full precision (W16A4), thus sacrificing inference efficiency.

3.3 Ablation Study

We conduct ablation studies on the Hadamard transformation in $\mathcal{H}$ -BitLinear at the 1.3B and 3B model scales. All models are trained on the same dataset to ensure a fair comparison. We report perplexity on the C4 validation set and zero-shot accuracy across a range of language tasks, including ARC-Easy [YBS19], ARC-Challenge [YBS19], Hellaswag [ZHB⁺19], Winogrande [SBBC20], PIQA [BZB⁺19], and LAMBADA [PKL⁺16]. As shown in Table 5, removing the rotary transformation leads to model divergence. Moreover, while applying the Hadamard transformation to both the weights and activations results in faster convergence, it achieves similar performance to applying it only to the activations as training progresses. Therefore, for simplicity, we apply the Hadamard transformation only to the activations in $\mathcal{H}$ -BitLinear. Detailed results can be found in Appendix A.

4 Conclusion

We introduce BitNet v2, enabling native 4-bit activations within 1-bit LLMs. This is achieved using our proposed $\mathcal{H}$ -BitLinear layer in place of standard attention output and FFN down projections. $\mathcal{H}$ -BitLinear employs an online Hadamard transformation before activation quantization to effectively

suppress outlier channels by reshaping the activation distribution. Our experiments show BitNet v2 with 8-bit activations matches BitNet b1.58 performance. Subsequently, BitNet v2 can be trained for native 4-bit activation use. This 4-bit variant, BitNet v2(a4), maintains comparable performance to the 8-bit version while significantly boosting efficiency in batched inference scenarios.

References

- [AMC⁺24] Saleh Ashkboos, Amirkeivan Mohtashami, Maximilian L. Croci, Bo Li, Martin Jaggi, Dan Alistarh, Torsten Hoefer, and James Hensman. Quarot: Outlier-free 4-bit inference in rotated llms. *CoRR*, abs/2404.00456, 2024.
- [BLC13] Yoshua Bengio, Nicholas Léonard, and Aaron C. Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *CoRR*, abs/1308.3432, 2013.
- [BZB⁺19] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: reasoning about physical commonsense in natural language. *CoRR*, abs/1911.11641, 2019.
- [Com23] Together Computer. Redpajama: an open dataset for training large language models, 2023.
- [GTA⁺24] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac’h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation, 07 2024.
- [LPC⁺24] James Liu, Pragaash Ponnusamy, Tianle Cai, Han Guo, Yoon Kim, and Ben Athiwaratkun. Training-free activation sparsity in large language models. *CoRR*, abs/2408.14690, 2024.
- [LZF⁺24] Zechun Liu, Changsheng Zhao, Igor Fedorov, Bilge Soran, Dhruv Choudhary, Raghuraman Krishnamoorthi, Vikas Chandra, Yuandong Tian, and Tijmen Blankevoort. Spinquant: LLM quantization with learned rotations. *CoRR*, abs/2405.16406, 2024.
- [MWM⁺24] Shuming Ma, Hongyu Wang, Lingxiao Ma, Lei Wang, Wenhui Wang, Shaohan Huang, Li Dong, Ruiping Wang, Jilong Xue, and Furu Wei. The era of 1-bit llms: All large language models are in 1.58 bits. *CoRR*, abs/2402.17764, 2024.
- [PKL⁺16] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Quan Ngoc Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The LAMBADA dataset: Word prediction requiring a broad discourse context. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics, ACL 2016, August 7-12, 2016, Berlin, Germany, Volume 1: Long Papers*. The Association for Computer Linguistics, 2016.
- [RSR⁺19] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *CoRR*, abs/1910.10683, 2019.
- [SBBC20] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: an adversarial winograd schema challenge at scale. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence*, pages 8732–8740, 2020.
- [Sha20] Noam Shazeer. GLU variants improve transformer. *CoRR*, abs/2002.05202, 2020.
- [WMD⁺23] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. Bitnet: Scaling 1-bit transformers for large language models. *CoRR*, abs/2310.11453, 2023.

- [WMW24] Hongyu Wang, Shuming Ma, and Furu Wei. Bitnet a4.8: 4-bit activations for 1-bit llms. *CoRR*, abs/2411.04965, 2024.
- [WMWW24] Hongyu Wang, Shuming Ma, Ruiping Wang, and Furu Wei. Q-sparse: All large language models can be fully sparsely-activated. *CoRR*, abs/2407.10969, 2024.
- [WZS⁺25] Jinheng Wang, Hansong Zhou, Ting Song, Shijie Cao, Yan Xia, Ting Cao, Jianyu Wei, Shuming Ma, Hongyu Wang, and Furu Wei. Bitnet. cpp: Efficient edge inference for ternary llms. *arXiv preprint arXiv:2502.11880*, 2025.
- [YBS19] Vikas Yadav, Steven Bethard, and Mihai Surdeanu. Quick and (not so) dirty: Unsupervised selection of justification sentences for multi-hop question answering. In Kentaro Inui, Jing Jiang, Vincent Ng, and Xiaojun Wan, editors, *EMNLP-IJCNLP*, 2019.
- [ZHB⁺19] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: can a machine really finish your sentence? In *Proceedings of the 57th Conference of the Association for Computational Linguistics*, pages 4791–4800, 2019.
- [ZS19] Biao Zhang and Rico Sennrich. Root mean square layer normalization. In Hanna M. Wallach, Hugo Larochelle, Alina Beygelzimer, Florence d’Alché-Buc, Emily B. Fox, and Roman Garnett, editors, *Advances in Neural Information Processing Systems*, pages 12360–12371, 2019.

A More results

Methods	Size	ARCC $\uparrow$	ARCE $\uparrow$	HS $\uparrow$	PQ $\uparrow$	WGe $\uparrow$	LBA $\uparrow$	Avg $\uparrow$
No rotation				diverged				
Weight & activation rotation	1.3B	27.13	50.29	48.32	69.04	54.30	53.76	50.47
Activation rotation		27.90	49.96	48.37	69.42	57.22	54.14	51.17
No rotation				diverged				
Weight & activation rotation	3B	30.03	55.72	56.81	71.65	59.43	59.65	55.54
Activation rotation		30.55	55.56	57.19	71.33	58.72	60.90	55.71

Table 6: Ablations on the Hadamard transformation of $\mathcal{H}$ -BitLinear across various sizes. All models have 1.58-bit weights and 8-bit activations.

Methods	Size	ARCC $\uparrow$	ARCE $\uparrow$	HS $\uparrow$	PQ $\uparrow$	WGe $\uparrow$	LBA $\uparrow$	Avg $\uparrow$
No rotation				diverged				
Weight & activation rotation	1.3B	27.22	49.12	47.77	69.37	54.54	52.49	50.09
Activation rotation		27.56	49.58	48.00	68.23	55.49	53.58	50.41
No rotation				diverged				
Weight & activation rotation	3B	29.44	54.46	56.57	71.93	57.85	59.64	54.98
Activation rotation		28.92	55.01	56.59	71.65	59.67	60.74	55.43

Table 7: Ablations on the Hadamard transformation of $\mathcal{H}$ -BitLinear across various sizes. All models have 1.58-bit weights and 4-bit activations.

B Hyper-parameters

Size	Hidden Size	GLU Size	#Heads	#Layers	Batch Size	# Tokens	Seq Length
400M	1024	4096	16	24	1M	100B	2048
1.3B	2048	8192	32	18	1M	100B	2048
3B	4096	8192	32	20	1M	100B	2048
7B	4096	16384	32	24	1M	100B	2048

Table 8: Model configurations for the BitNet models.

Model	Size	Learning Rate	Weight Decay	Warm-up	Adam β
BitNet	400M	$1.8 \times 10^{-3} \rightarrow 1.2 \times 10^{-3}$	$0.1 \rightarrow 0$	375	(0.9, 0.95)
	1.3B	$1.2 \times 10^{-3} \rightarrow 8 \times 10^{-4}$	$0.1 \rightarrow 0$	375	(0.9, 0.95)
	3B	$1.2 \times 10^{-3} \rightarrow 6.4 \times 10^{-4}$	$0.1 \rightarrow 0$	375	(0.9, 0.95)
	7B	$1 \times 10^{-3} \rightarrow 6 \times 10^{-4}$	$0.1 \rightarrow 0$	375	(0.9, 0.95)

Table 9: Hyper-parameters for both BitNet v2 training.